

小结

本章介绍了一些特殊 IO 操作和四个标准库类型：tuple、bitset、正则表达式和随机数。

tuple 是一个模板，允许我们将多个不同类型的成员捆绑成单一对象。每个 tuple 包含指定数量的成员，但对一个给定的 tuple 类型，标准库并未限制我们可以定义的成员数量上限。

bitset 允许我们定义指定大小的二进制位集合。标准库不限制一个 bitset 的大小必须与整型类型的大小匹配，bitset 的大小可以更大。除了支持普通的位运算符（参见 4.8 节，第 136 页）外，bitset 还定义了一些命名的操作，允许我们操纵 bitset 中特定定位的状态。

正则表达式库提供了一组类和函数：regex 类管理用某种正则表达式语言编写的正则表达式。匹配类保存了某个特定匹配的相关信息。这些类被函数 regex_search 和 regex_match 所用。这两个函数接受一个 regex 对象和一个字符序列，检查 regex 中的正则表达式是否匹配给定的字符序列。regex 迭代器类型是迭代器适配器，它们使用 regex_search 遍历输入序列，返回每个匹配的子序列。标准库还定义了一个 regex_replace 函数，允许我们用指定内容替换输入序列中与正则表达式匹配的部分。

随机数库由一组随机数引擎类和分布类组成。随机数引擎返回一个均匀分布的整型值序列。标准库定义了多个引擎，它们具有不同的性能特点。default_random_engine 是适合于大多数普通情况的引擎。标准库还定义了 20 个分布类型。这些分布类型使用一个引擎来生成指定类型的随机数，这些随机数的值都在给定范围内，且分布满足指定的概率分布。

术语表

bitset 标准库类，保存二进制位集合，大小在编译时已知，并提供检测和设置集合中二进制位的操作。

cmatch **csub_match** 对象的容器，保存一个 regex 与一个 const char* 输入序列匹配的相关信息。容器首元素描述了整个匹配结果。后续元素描述了子表达式的匹配结果。

cregex_iterator 类似 sregex_iterator，唯一的差别是此迭代器遍历一个 char 数组。

csub_match 保存一个正则表达式与一个 const char* 匹配结果的类型。可以表示整个匹配或子表达式的匹配。

默认随机数引擎 (default random engine) 用于普通用途的随机数引擎的类型别名。

格式化 IO (formatted IO) 读写操作，利用要读写的对象的类型来定义操作的行为。格式化输入操作执行适合要读取的类型的转换操作，如将 ASCII 码字符串转换为算术类型以及（默认地）忽略空白符。格式化输出操作将类型转换为可打印的字符表示形式、补白输出，还可能执行其他与输出类型相关的转换。

get 模板函数，返回给定 tuple 的指定成员。例如，get<0>(t) 返回 tuple 的第一个成员。

高位 (high-order) bitset 中下标最大的那些位。

低位 (low-order) bitset 中下标最小的那些位。

操纵符 (manipulator) “操纵”流的类函数对象。操纵符可用作重载的 IO 运算符<<和>>的右侧运算对象。大多数操纵符会改变流对象的内部状态。这种操纵符通常是成对的——一个改变状态，另一个恢复到流的默认状态。

随机数分布 (random-number distribution) 标准库类型，根据其名字所指出的概率分布转换随机数引擎的输出值。例如，`uniform_int_distribution<T>`生成类型为 `T` 的均匀分布的整数，而 `normal_distribution<T>`生成正态分布的值，依此类推。

随机数引擎 (random-number engine) 标准库类型，生成随机的无符号数。引擎的设计意图是只用作随机数分布的输入。

随机数发生器 (random-number generator) 一个随机数引擎类型和一个分布类型的组合。

regex 管理正则表达式的类。

regex_error 异常类型，当正则表达式中存在语法错误时抛出此异常。

regex_match 确定整个输入序列是否与给定 `regex` 对象匹配的函数。

regex_replace 使用一个 `regex` 对象来匹配输入序列并用给定格式替换匹配的子表达式的函数。

regex_search 使用一个 `regex` 对象在给定输入序列中查找匹配的子序列的函数。

正则表达式 (regular expression) 一种描述字符序列的方式。

种子 (seed) 提供给随机数引擎的值，使引擎移动到生成的随机数序列中一个新的点。

smatch `ssub_match` 对象的容器，提供一个 `regex` 与一个 `string` 输入序列匹配的相关信息。容器首元素描述了整个匹配结果。后续元素描述了子表达式的匹配结果。

sregex_iterator 迭代器，使用给定的 `regex` 对象遍历一个 `string` 来查找匹配子串。其构造函数通过调用 `regex_search` 将迭代器定位到第一个匹配。递增迭代器的操作会调用 `regex_search`，从给定 `string` 中当前匹配之后的位置开始查找匹配。解引用迭代器返回一个描述当前匹配的 `smatch` 对象。

ssub_match 保存正则表达式与 `string` 匹配结果的类型。可以描述整个匹配或子表达式的匹配。

子表达式 (subexpression) 正则表达式模式中用括号包围的组成部分。

tuple 模板，生成的类型保存指定类型的未命名成员。标准库没有限制一个 `tuple` 最多可以包含多少个成员。

未格式化 IO (unformatted IO) 将流当作无差别的字节流来处理的操作。未格式化操作给用户增加了很多管理 IO 的负担。

第 18 章

用于大型程序的工具

内容

18.1 异常处理.....	684
18.2 命名空间.....	695
18.3 多重继承与虚继承	710
小结	722
术语表.....	722

C++语言能解决的问题规模千变万化，有的小到一个程序员几小时就能完成，有的则是含有几千几万行代码的庞大系统，需要几百个程序员协同工作好几年。本书之前介绍的内容对各种规模的编程问题都适用。

除此之外，C++语言还包含其他一些特征，当我们编写比较复杂的、小组和个人难以管理的系统时，这些特征最为有用。本章的主题即是向读者介绍这些特征，它们包括异常处理、命名空间和多重继承。